Global Warming Science

https://courses.seas.harvard.edu/climate/eli/Courses/EPS101/

Floods!

Please use the template below to answer the workshop questions. "XX" indicates places where you need to complete/write code or add a discussion.

Your name:

[ ]:
```
# Load libraries and data:
import numpy as np
import pandas as pd
import pickle
import matplotlib.pyplot as plt
from scipy import stats
from scipy.integrate import solve_ivp
from scipy import signal
from scipy import optimize
import warnings
import cartopy.crs as ccrs
from datetime import datetime
from matplotlib.dates import DayLocator, DateFormatter
from matplotlib.ticker import MultipleLocator, NullFormatter
from matplotlib.dates import DayLocator, DateFormatter
from mpl_toolkits.axes_grid1.inset_locator import inset_axes
from matplotlib.dates import DayLocator, DateFormatter


# load the data from a pickle file:
with open('./floods_variables.pickle', 'rb') as file:
    while 1:
        try:
            d = pickle.load(file)
        except (EOFError):
            break
            # print information about each extracted variable:
        for key in list(d.keys()):
            if key != "precipitation_station_data_dictionary":
                print("extracting pickled variable: name=", key\
                      ,  "; size=", d[key].shape)
        globals().update(d)


# Some of the variables are dictionaries/list of dictionaries and needs to be further␣
 ↪extracted:
precipitation_station_data_dictionary=precipitation_station_data_dictionary[()]
hurricane_Harvey_flood_data=hurricane_Harvey_flood_data[()]
tide_gauge_records=tide_gauge_records[()]

print("done.")
```

Explanation of variables

hurricane Harvey flood

hurricane_Harvey_flood_data is a dictionary containing precipitation data, water level data, and the defined flood thresholds for the Houston area. print its keys to see more.

storm surges and coastal floods

tide_gauge_records is a list of a few elements; each element is a dictionary with the relevant information for a given coastal tide event. Print tide_gauge_records[0].keys to see more.

analyzing extreme rain events in weather station data:

precipitation_station_data_dictionary is a dictionary with precipitation time series from several weather stations. The keys are weather station names, and the values are the dates and corresponding daily precipitation values.

analyzing projections extreme precipitation events

extreme_precip_1920_1940: value of 99.9% precentile precipitation as function of latitude and ensemble member.

extreme_precip_2080_2100: same, for end of 21st century

extreme_precip_TS_1920_1940: surface temperature vs latitude

extreme_precip_TS_2080_2100: same

Data of precipitation every day for 20 years as a function of time (7300 days) and longitude (288 longitudes), at one latitude (40N):

extreme_precip_daily_precip_40N_1920_1940

extreme_precip_daily_precip_40N_2080_2100

extreme_precip_lat

analyzing projected changes in global precipitation patterns

10-year average of precipitation vs latitude and longitude, for preindustrial and the ssp3-70 scenario:

precipitation_vs_lat_lon_1850, precipitation_vs_lat_lon_2085

latitude and longitude axes:

precipitation_lat, precipitation_lon

analyzing the wet getting wetter, dry getting drier projection:

wet_wetter_E1920: zonally-averaged, time-averaged evaporation (in m/s) vs latitude for 1920-1940

wet_wetter_E2080: same, for 2080-2100

wet_wetter_P1920: same, precipitation (in m/s)

wet_wetter_P2080: same

wet_wetter_TS1920: zonally-averaged, time averaged, surface temperature for 1920-1940

wet_wetter_TS2080: for 2080-2100

wet_wetter_lat: latitudes for above data

1. The Hurricane Harvey Houston floods.

Plot the daily precipitation in Houston for 2000–2025. Plot the water level during August 23 to September 4, 2017 and the water level variations predicted due to tidal variations. Calculate the total precipitation during this event and compare it to the total annual precipitation in the same station averaged over the 25-year period.

[ ]:
```
fig, ax = plt.subplots(nrows=2, ncols=1, figsize=(8, 6),dpi=300)

# Plot daily precipitation over time:
# -----------------------------------

x=hurricane_Harvey_flood_data["precipitation-time"]
y=XX
ax[0].plot(x, y,linewidth=1.0)
start = pd.Timestamp("2000-01-01")
end   = pd.Timestamp(XX)
ax[0].set_xlim(start, end)
ax[0].set_title("XX")
ax[0].set_xlabel("XX")
ax[0].set_ylabel("Precipitation (mm/day)")
ax[0].grid(True, alpha=0.3)


# Optional extra credit: Plot inset with August/September 2017 daily precipitation:
axins = inset_axes(
    ax[0],
    width="XX%", height="XX%",
    loc="upper left",                  # still anchored to the top-left
    bbox_transform=ax[0].transAxes,    # interpret bbox_to_anchor in axes coords
    borderpad=1.0
)

axins.bar(x, y, width=1.0, align="center")
axins.set_title(XX)
start = pd.Timestamp("XX")
end   = pd.Timestamp("XX")
axins.set_xlim(start, end)



# Plot water level, predicted vs observed:
# ----------------------------------------
time = hurricane_Harvey_flood_data["tide-gauge-time"]
predicted = XX
observed = XX
thresholds = XX
ax[1].plot(XX, XX, label="Predicted (m)", linewidth=1.0)
ax[1].plot(XX, XX, label="Observed (m)", linewidth=1.0)

# add horizontal dash lines for floods thresholds:
ax[1].plot(XX)
ax[1].plot(XX)
ax[1].plot(XX)
```

```
ax[1].set_title("XX")
ax[1].grid(which='both', alpha=0.3)
ax[1].set_xlabel("XX")
ax[1].set_ylabel("XX")
```

2. Storm surges, tides, and coastal floods.

Plot the expected (predicted) tidal variation in sea level, and the actual sea level during five separate storm surge events. Superimpose horizontal lines showing the thresholds defining minor, moderate, and major flood levels. Note the large variability in tidal amplitudes in the different locations and describe the role of tides in the occurrence of storm surges in the specific plotted records.

[ ]:
```
fig, ax = plt.subplots(nrows=len(tide_gauge_records), ncols=1, figsize=(6, 8), dpi=300)
for ifile in range(len(tide_gauge_records)):
    # Plot water level, predicted vs observed:
    time=tide_gauge_records[ifile]["time"]
    predicted=tide_gauge_records[ifile]["XX"]
    observed=tide_gauge_records[ifile]["XX"]
    ax[ifile].plot(XX, XX, label="Predicted (m)", linewidth=1.0)
    ax[ifile].plot(XX, XX, label="Observed (m)", linewidth=1.0)

    # add dash lines for floods thresholds:
    # linestyle (offset, (on, off, on, off, ...))
    thresholds=tide_gauge_records[ifile]["flood-thresholds"]
    ax[ifile].plot(XX)
    ax[ifile].plot(XX)
    ax[ifile].plot(XX)

    ax[ifile].set_title(tide_gauge_records[ifile]["title"])
    ax[ifile].legend()

    ax[ifile].set_ylabel("Sea level (m)")
    ax[ifile].grid(True, alpha=0.3)

    # make every subplot show month/day for each day, no year, no time
    ax[ifile].xaxis.set_major_locator(DayLocator(interval=1))
    ax[ifile].xaxis.set_major_formatter(DateFormatter("%m/%d"))
    ax[ifile].tick_params(axis='x', which='major', labelbottom=True)


plt.tight_layout()
```

Discussion.

3. Analyzing extreme precipitation in weather station data:

(a) For each given weather station, draw a scatter plot of precipitation as a function of time starting at 1880. Add a histogram of values plotted as a probability distribution function for the first 20 years and for the last 20 years of the record. Discuss the difficulty of analyzing the statistics of intense rain events using a short observed record.

[ ]:
```
# Scatter plot station precipitation time series:

num_keys = len(precipitation_station_data_dictionary)
```

```
fig, axs = plt.subplots(num_keys, 2, figsize=(7, 2 * num_keys),dpi=600)#, sharex=True)
nyears_hist=20 # number of years to histogram at the beginning and end of record


# Iterate through each key-value pair in the dictionary
for i, (key, values) in enumerate(precipitation_station_data_dictionary.items()):

    station_name=key
    datetimes = values[0][:]
    precipitation_values = values[1][:]

    # Plot the time series:
    # ---------------------
    axs[i,0].scatter(XX, XX
                     ,marker='x',linewidth=0.2,s=12
                     ,label=station_name)
    axs[i,0].set_title(station_name)
    axs[i,0].set_ylabel('Precipitation (mm)')

    # Add labels and legend:
    axs[i,0].set_xlim(datetime(1880,1,1),datetime(2022,12,31))

    # Plot the PDF as a bar plot:
    # ---------------------------
    x=np.asarray(precipitation_station_data_dictionary[station_name][1])
    x[np.isnan(x)]=0.0
    x1=x[:XX]    # first nyears_hist years
    x2=x[XX:]    # last nyears_hist years
    xmax=XX # largest element in both periods
    xmin=XX
    hist_x1, bin_edges_x1 = np.histogram(x1, bins=50, density=True, range=[xmin,xmax])
    hist_x2, bin_edges_x2 = np.histogram(XX)
    hist_x1[0]=0
    hist_x2[0]=0
    # Calculate bin centers
    bin_centers = (bin_edges_x1[:-1] + bin_edges_x1[1:]) / 2

    # Plot the PDF using bar

    width=bin_edges_x1[1] - bin_edges_x1[0]
    label=XX # year range used
    axs[i,1].bar(bin_centers, hist_x1, width=width, alpha=0.7, color='blue'
            , edgecolor='blue',label=label)
    label=XX
    axs[i,1].bar(bin_centers, hist_x2, width=width, alpha=0.7, color='red'
            , edgecolor='red',label=label)

    axs[i,1].set_title(XX)
    axs[i,1].set_ylabel(XX)
    axs[i,1].legend()
    axs[i,1].set_xlim(0,bin_edges_x1[-1])
    axs[i,1].set_yscale('log')
```

```
axs[-1,0].set_xlabel(XX)
axs[-1,1].set_xlabel(XX)

# Adjust layout and show the plot
plt.tight_layout()
```

Discussion: XX

(b) Create a normally-distributed random data set for a variable x with a zero mean and std of 1, and with the number of points equal to $365 \times 20 \times 20 \times 10^i$, where $i = 0, 1, 2, 3$. Plot the probability distribution function histogram $PDF(x)$ for $-4 < x < 4$ in each case on a log y-axis. Discuss how the estimate of the tail of the distribution improves with an increasing number of data points.

[ ]:
```
plt.figure(dpi=300)
for i in range(0,4):
    N=365*20*10**i
    data=np.random.normal(loc=0.0, scale=1.0, size=N)
    plt.subplot(4,1,i+1)
    hist, bin_edges = np.histogram(data, bins=150, density=True, range=[-4,4])
    bin_centers = XX
    width=XX
    h=plt.bar(XX, XX, width=width)
    plt.yscale('log')
    plt.title("N=%g, or %g years" % (N,20*10**i) )
    plt.ylabel(XX)
    if i==3:
        plt.xlabel(XX)

plt.tight_layout()
```

Discussion: XX

4. Projected extreme precipitation events:

(a) Plot the PDF of daily precipitation at 40°N for 1920–1940 and for 2080–2100 under RCP8.5. Calculate the 99.9th percentile precipitation rate (mm/day) for each period.

[ ]:
```
# calculate probability distribution function of precipitation, and then the CDF:
lat=extreme_precip_lat
Nbin_edges40N=50
bin_edges40N=np.arange(0,140.0,140.0/Nbin_edges40N)
# calculate distribution at one latitude:
hist,bin_edges_out=np.histogram(extreme_precip_daily_precip_40N_1920_1940,bin_edges40N\
                            ,density=True,range=(0,140))
distribution_40N_1920_1940=hist
dP=bin_edges_out[2]-bin_edges_out[1]
xaxis_from_bin_edges_40N_1920_1940=bin_edges_out[0:-2]+dP/2

# repeat for 2080-2100:
hist,bin_edges_out=np.histogram(XX)
distribution_40N_2080_2100=XX
dP=XX
xaxis_from_bin_edges_40N_2080_2100=XX

# ========================================================================
```

```
# plot precipitation distribution at 40N:
# ============================================================================
plt.figure(figsize=(6,3),dpi=300)
plt.semilogy(xaxis_from_bin_edges_40N_1920_1940,distribution_40N_1920_1940[1:
  ↪],"-",color="b",lw=0.5,label="1920-1940")
plt.semilogy(XX,distribution_40N_2080_2100[1:],"-",color="r",lw=0.5,label="2080-2100")
plt.legend()
plt.xlabel("Precipitation (mm/day)");
plt.ylabel("Probability");
plt.grid(lw=0.25)
```

(b) (i) Plot the 99.9th percentile precipitation rate as a function of latitude for the two periods. (ii) Plot the temperature corresponding to the 99.9th percentile precipitation rate vs. latitude for the two periods.

[ ]:
```
# average extreme precipitation over ensemble members:
extreme_precip_avg_1920_1940=np.mean(extreme_precip_1920_1940,axis=XX)
extreme_precip_avg_2080_2100=XX


# ============================================================================
# plot extreme precipitation:
# ============================================================================
plt.figure(figsize=(4,3),dpi=200)
# first period:
plt.plot(lat,extreme_precip_avg_1920_1940.flatten(),lw=1,color="b")
# second period:
plt.plot(XX...,color="r")
plt.xlabel("XX")
plt.ylabel("XX")
```

(c) Optional extra credit: Plot the projected percent change per degree Kelvin warming of the 99.9th percentile precipitation rate as a function of latitude based on the above curves. Superimpose the expected percent change in extreme precipitation per degree Kelvin based on the expected change to the surface saturation specific humidity (that is, based on the Clausius-Clapeyron scaling).

Discuss the justification for using the Clausius-Clapeyron scaling to predict changes in extreme precipitation and what factors may lead to the deviation from that scaling in the above plot.

[ ]:
```
# parameters needed below:
p_s=1013
dT_warming=1


# average TS corresponding to extreme precipitation over ensemble members:
extreme_precip_avg_TS_1920_1940=XX
extreme_precip_avg_TS_2080_2100=XX
# Change in temperature during extreme precipitation, as a function of latitude:
Delta_TS=extreme_precip_avg_TS_2080_2100-extreme_precip_avg_TS_1920_1940

# calculate CDFs to be used to find the level of precipitation corresponding to 99.9␣
  ↪percentile events.
dP=bin_edges40N[1]-bin_edges40N[0] # bin width
```

```
# normalize CDFs such that they go from 0 to 100%:
CDF_40N_1920_1940=100*np.cumsum(distribution_40N_1920_1940)*dP
CDF_40N_2080_2100=XX

# find precipitation rate corresponding to specified percentile:
percentile=99.9
# first for 1920-1940:
i1=np.argmax(CDF_40N_1920_1940>percentile) # first index for which condition occurs
# prepare for interpolating to find the exact precipitation rate corresponding to
# desired percentile by finding the bin edges on the two sides of the desired␣
 ↪percentile:
PRECT1=bin_edges40N[i1-1]+dP/2
PRECT2=bin_edges40N[i1]+dP/2
CDF1=CDF_40N_1920_1940[i1-1]
CDF2=CDF_40N_1920_1940[i1]
# interpolate:
PRECT_percentile_1920_1940=PRECT1+(PRECT2-PRECT1)*(percentile-CDF1)/(CDF2-CDF1)

# repeat for 2080-2100:
XX

# ------------------------------------------------------------------------
# first, plot surface temperature vs latitude, TS, for the two periods:
# ------------------------------------------------------------------------
plt.figure(figsize=(6,6),dpi=300)
plt.subplot(2,2,1)

plt.plot(XX,color="b",label="$T_s$ 1920-1940")
plt.plot(XX,color="r",label="$T_s$ 2080-2100")
plt.xlabel("XX")
plt.ylabel("XX")


# ------------------------------------------------
# plot precipitation distribution at one latitude:
# ------------------------------------------------
plt.subplot(2,2,2)

plt.plot(XX,CDF_40N_1920_1940,color="b",lw=0.5)
plt.plot(XX,CDF_40N_2080_2100,color="r",lw=0.5,alpha=0.5)
plt.axhline(percentile,color="k",lw=0.5,alpha=0.5)
# vertical bars at specified percentile value for each of the two periods:
plt.axvline(XX,color="b",lw=1)
plt.axvline(XX,color="r",lw=1)
plt.xlabel("XX")
plt.ylabel("XX")


# ------------------------------------------------------------------------
# Helper functions for calculating fractional extreme precipitation
# change per degree warming:
# ------------------------------------------------------------------------
def q_sat(T,P):
```

```
    # saturation specific humidity (gr water vapor per gram moist air):
    # inputs:
    # T: temperature, in K
    # P: pressure, in mb

    R_v = 461 # Gas constant for moist air = 461 J/(kg*K)
    R_d = 287 # Gas constant 287 J K^-1 kg^-1
    TT = T-273.15 # Kelvin to Celsius
    # Saturation water vapor pressure (mb) from Emanuel 4.4.14 p 116-117:
    ew = 6.112*np.exp((17.67 * TT) / (TT + 243.5))
    # saturation mixing ratio (gr water vapor per gram dry air):
    rw = (R_d / R_v) * ew / (P - ew)
    # saturation specific humidity (gr water vapor per gram moist air):
    qw = rw / (1 + rw)
    return qw


def calc_dq_sat_dT_CC_percent(TS):
    # CC scaling:
    dq_sat_dT_percent=0.0*TS
    i=-1
    for T in TS:
        i=i+1
        # calculate 100*(dq^*/dT)/q^*
        dq_sat_dT_percent[i]=XX
    return dq_sat_dT_percent

# ------------------------------------------------------------------------
# plot fractional extreme precipitation change per degree warming:
# ------------------------------------------------------------------------
plt.subplot(2,2,3)

mygray=[0.7, 0.7, 0.7]
Delta_extreme_TS=extreme_precip_avg_TS_2080_2100-extreme_precip_avg_TS_1920_1940
DeltaP_percent=100*(extreme_precip_avg_2080_2100-extreme_precip_avg_1920_1940) \
        /extreme_precip_avg_2080_2100
plt.plot(lat,DeltaP_percent/Delta_extreme_TS,color="r",lw=1,label="RCP8.5")
dq_sat_dT_CC_percent=calc_dq_sat_dT_CC_percent(extreme_precip_avg_TS_2080_2100)
plt.plot(lat,dq_sat_dT_CC_percent,color=mygray,linestyle="-",label="C-C")
plt.ylabel("Precipitation change (% K$^{-1}$)");
plt.xlabel("Latitude");
plt.legend()

plt.tight_layout(pad=0);
```

Discussion: XX

(d) Optional extra credit: Plot the Clausius-Clapeyron scaling for the expected increase in extreme precipitation rates with temperature, based on the dependence of the saturation specific humidity on temperature T, from 230 K to 320 K. Superimpose the dependence of the thermodynamical component of equation 12.3, normalized to have the same value as the Clausius-Clapeyron scaling at T =230 K. Discuss the difference between the two estimates. Why are these estimates less relevant in the tropics?

[ ]:
```
# calculate two scaling for a linear range of temperatures:
# ----------------------------------------------------------

# saturation specific humidity:
def q_sat(T,P):
    # saturation specific humidity (gr water vapor per gram moist air):
    # inputs:
    # T: temperature, in K
    # P: pressure, in mb

    R_v = 461 # Gas constant for moist air = 461 J/(kg*K)
    R_d = 287 # Gas constant 287 J K^-1 kg^-1
    TT = T-273.15 # Kelvin to Celsius
    # Saturation water vapor pressure (mb) from Emanuel 4.4.14 p 116-117:
    ew = 6.112*np.exp((17.67 * TT) / (TT + 243.5))
    # saturation mixing ratio (gr water vapor per gram dry air):
    rw = (R_d / R_v) * ew / (P - ew)
    # saturation specific humidity (gr water vapor per gram moist air):
    qw = rw / (1 + rw)
    return qw


def calc_dq_sat_dp_MSE(T):
    # calc dq_sat/dp at a constant MSE as function of T.
    # Input: array T.
    dq_sat_dp_MSE=T*0.0
    i=-1
    for T in TS:
        i=i+1
        dp=-1 # hPa
        dz=XX # height difference corresponding to dp
        q_s=q_sat(T,p_s) # surface specific humidity, assumed saturated
        MSE_s = XX
        # calculate T_parcel, raised adianatically from p_s to p_s+dp
        XX
        dT=T_parcel-T
        # use finite differencedd to calculate teh following two:
        dqdp=XX
        dqdT=XX
        dTdp_MSE=dT/dp
        dq_sat_dp_MSE[i]=XX

    return dq_sat_dp_MSE

def MSE_conservation(T,P,z,MSE_s,q_s):
    """ This function is called by a root finder to calculate
    the temperature of a rising parcel. it is equal to zero when
    MSE conservation is satisfied.
    """
    qsat=q_sat(T,P)
    # the root finder is looking for the value of T for which the following is zero:
    ans = cp_dry*T+L*min(q_s,qsat)+g*z-MSE_s
    return(ans)
```

```
Tbar=260 # needed for exponential presure, to convert between z and p.
g=9.81        # gravity
L=24.93e5     # latent heat J/kg (vaporization at t.p.)
cp_dry=1005   # J/kg/K
T1=np.arange(232,330,1)
R=287 # Gas constant 287 J K^-1 kg^-1
# calculate q_sat(T1) for
q_sat_T1=0.0*T1
i=-1
for T in T1:
    i=i+1
    q_sat_T1[i]=XX
dq_sat_dp_MSE_T1=calc_dq_sat_dp_MSE(T1)



# plot the two scalings:
fig=plt.figure(figsize=(5,3),dpi=300)

plt.semilogy(XX,XX,label="$q^*(T)$")
plt.semilogy(XX,XX,label="$\\left.\\frac{dq^*}{dp~}\\right|_{\\rm MSE}$")

plt.legend()
plt.xlim(230,320)
plt.grid(lw=0.25)
plt.xlabel("Temperature (K)")
plt.tight_layout();
```

5. projected changes in global precipitation patterns:

Contour the precipitation pattern for the preindustrial period, the year 2100 based on the SSP3-7.0 projection, and the difference between the two.

[ ]:
```
# plot for the preindustrial:
# ---------------------------
fig=plt.figure(figsize=(8,4))
ax = plt.axes(projection=ccrs.PlateCarree(central_longitude=0.0, globe=None))
ax.set_extent([-180, 180, -90, 90], crs=ccrs.PlateCarree())
ax.coastlines(resolution='110m')
ax.gridlines()

delta=0.2
mylevels=np.arange(0,5+delta,delta)
to_m_per_year=365*24*3600

c=plt.contourf(precipitation_lon, precipitation_lat
               ,precipitation_vs_lat_lon_1850*to_m_per_year
               ,levels=mylevels)
# draw the colorbar
clb=plt.colorbar(c, shrink=0.85, pad=0.02)
# add title/ labels:
```

```
plt.xlabel('Longitude')
plt.ylabel('Latitude')
clb.set_label('m/year')
plt.title('Precipitation, 1850-1860')
plt.show()

# plot for 2085:
# --------------
fig=plt.figure(figsize=(8,4))
XX

# plot the difference: (change the contour levels, use the bwr color map):
# ------------------------------------------------------------------------
fig=plt.figure(figsize=(8,4),dpi=200)
XX
```

6. "Wet getting wetter, dry getting drier" projections:

(a) Plot the zonally averaged precipitation minus evaporation as a function of latitude for 1920–1940 and for 2080–2100 under RCP8.5.

(b) Plot the projected change in zonally averaged precipitation minus evaporation as a function of latitude, and superimpose the estimated expected change based on the Clausius-Clapeyron scaling in equation (12.2).

[ ]:
```
# clausius clapeyron: moisture (kg/kg) over deg K
alpha=0.07

# plot:
PmE1920=wet_wetter_P1920-wet_wetter_E1920
PmE2080=XX
deltaPmE=PmE2080-PmE1920
deltaTS=XX
# "Wet getting wetter, dry getting drier" scaling here:
deltaPmE_estimate=XX
lat=wet_wetter_lat

fig=plt.figure(dpi=300)

# plot surface temperature vs latitude for the two periods
plt.subplot(2,2,1)
XX

# plot warming as a function of latitude:
plt.subplot(2,2,2)
XX


# plot precipitation-evaporation for the two periods in units of mm/day
# (convert precipitation and evaporation from m/s to mm/day)
plt.subplot(2,2,3)
XX

# plot actual change in precipitation-evaporation and superimpose the CC-scaling
```

```
plt.subplot(2,2,4)
XX

plt.tight_layout()
```

7. Optional extra credit: Composite analyses of drought or flood years:

(a) Analyze the provided results from a long control run of a climate model to calculate and contour the rainy season averages (composites) of sea level pressure and precipitation anomalies for times in which the area-mean precipitation over the region of your city of birth is 1 std above the average. Describe the pattern you calculate and discuss the expected impacts on agriculture and flood risk for that region.

(b) Plot, describe, and explain the precipitation over the tropical Pacific, from Australia to South America, averaged over times corresponding to El Niño versus La Niña events. Define the times of these events as the NINO3.4 index being above and below 1 std.

[ ]:
```
# reproduce something like textbook Figure 12.2
XX
```

8. Guiding questions to be addressed in your report:

(a) What are the types of floods? Why are floods important to understand and predict?

(b) Why and how can each type change due to natural climate variability? Due to anthropogenic climate change? Due to other anthropogenic effects?

(c) Define extreme precipitation events. What are their socioeconomic consequences? How and why are they expected to change?

(d) Why are changes to the probability of rare precipitation events difficult to estimate using observations? Why are rare flood events difficult to attribute to anthropogenic climate change?

(e) What can we say about the projections of changes to global-scale time-mean precipitation patterns in a warmer climate? What about regional changes?

[ ]:
```

```